



Học Sâu (Deep Learning)

Chương 4: Bắt đầu với Mạng Nơ-ron (Phân loại và Hồi quy)

Đại học Đông Á | August 14, 2026

Nội dung chương

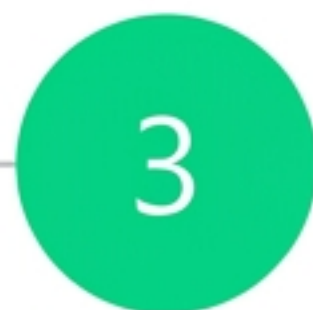
1. Các thuật ngữ
cốt lõi
(Glossary)



2. Phân loại Nhị
phân (IMDb
Dataset)



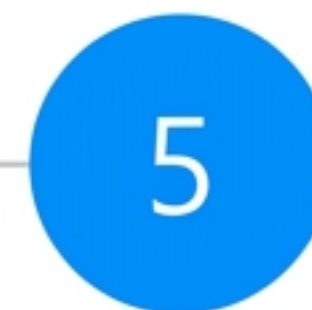
3. Phân loại Đa
lớp (Reuters
Dataset)



4. Hồi quy Vô
hướng (Boston
Housing)



5. Tổng kết



5. Tổng kết

Khái niệm Cơ bản

Các thuật ngữ nghiêm ngặt trong phân loại và hồi quy:



Sample (Mẫu / Đầu vào)

Một điểm dữ liệu đi vào mô hình. (Ví dụ: Một bức ảnh, đoạn văn bản).



Target (Mục tiêu / Sự thật)

Kết quả kỳ vọng. Mô hình lý tưởng sẽ dự đoán ra kết quả này theo tập chuẩn.



Prediction (Dự đoán / Đầu ra)

Giá trị đầu ra thực tế từ mô hình của bạn.



Loss value (Giá trị mất mát)

Khoảng cách đo lường sự chênh lệch giữa Prediction và Target.

Thuật ngữ về Nhãn (Labels) & Lô (Batches)



Classes (Lớp)

Tập hợp danh mục khả dĩ. (Ví dụ: Chó, Mèo, Chim).



Label (Nhãn)

Phiên bản cụ thể của Class gán cho Sample. (Ví dụ: Ảnh có nhãn "Chó").



Ground-truth (Sự thật gốc)

Toàn bộ targets được thu thập và gán nhãn thủ công.

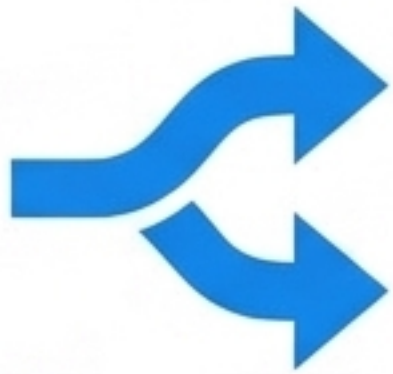


Batch (Mini-batch - Lô)

Tập hợp nhỏ samples (8-128) xử lý cùng lúc để tính gradient. Kích thước là lũy thừa của 2 để tối ưu bộ nhớ GPU.

Phân loại các bài toán Học máy cơ bản

3 bài toán phổ biến nhất trên dữ liệu dạng vector (bảng):



Phân loại Nhị phân (Binary)

Phân loại đầu vào thành 2 nhóm độc quyền.

Chó hoặc Mèo, Positive hoặc Negative.



Phân loại Đa lớp (Multiclass)

Phân loại thành nhiều hơn 2 nhóm (Single-label nếu chỉ thuộc 1 danh mục).

Nhận dạng chữ số 0-9.



Hồi quy Vô hướng (Scalar Regression)

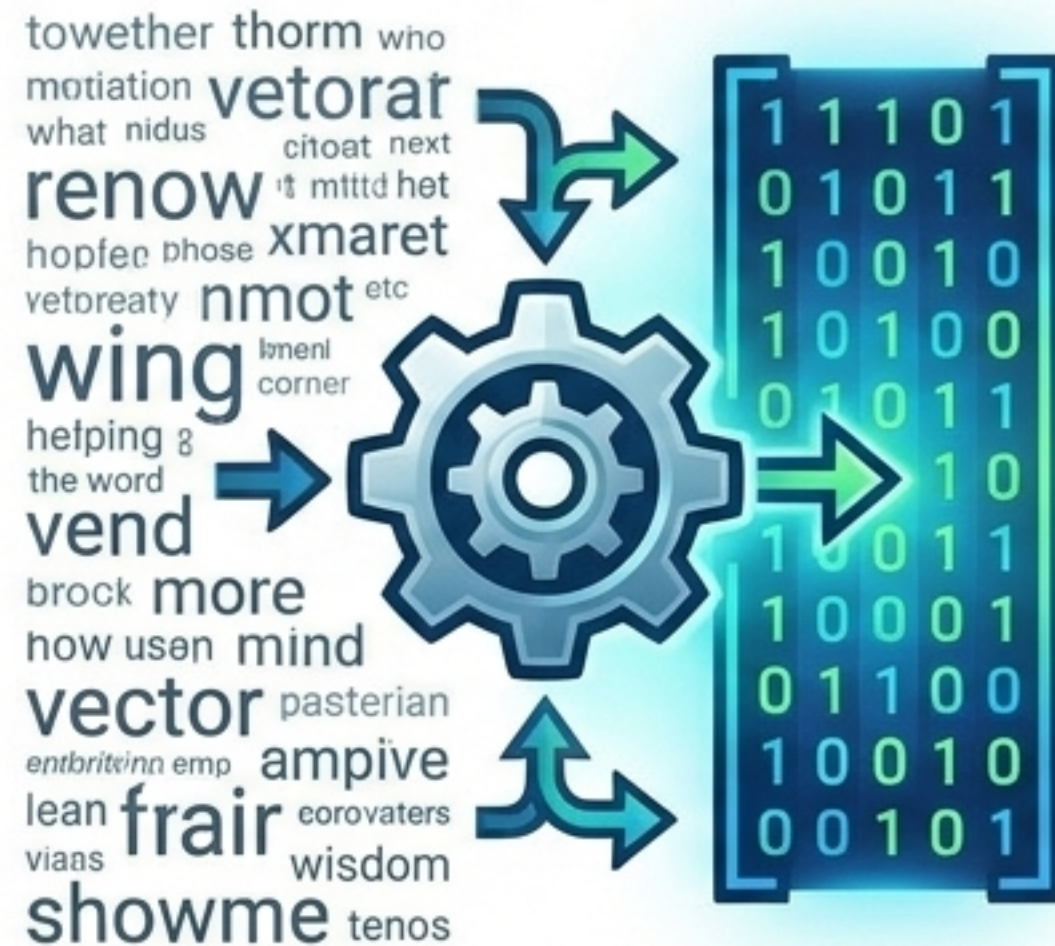
Dự đoán một giá trị mục tiêu liên tục (vô hướng).

Dự đoán giá nhà, nhiệt độ.

Chuẩn bị dữ liệu (Data Preprocessing)

Vấn đề

Mạng nơ-ron không nhận dữ liệu thô (chuỗi văn bản có độ dài khác nhau). Chúng ta phải biến danh sách từ vựng thành tensor.



Giải pháp

Mã hóa đa điểm (Multi-hot Encoding).

- Biến danh sách thành các vector 0 và 1 (chỉ số xuất hiện).
- Ví dụ: [8, 5] thành vector toàn 0, ngoại trừ chỉ số 5 và 8 là 1.

Mã hóa đa điểm (Mã Code)



```
import numpy as np
```

```
def multi_hot_encode(sequences, num_classes):  
    results = np.zeros((len(sequences), num_classes))  
    for i, sequence in enumerate(sequences):  
        results[i, sequence] = 1.0 # Dat 1 tai cac vi tri tu xuat hien  
    return results
```

Bài toán Đánh giá Cảm xúc (IMDb Dataset)



Mục tiêu

Đánh giá xem một bài bình luận phim là Tích cực (Positive - 1) hay Tiêu cực (Negative - 0).



Dữ liệu

50.000 bài đánh giá.
Chia đều 25.000 train và 25.000 test.
Cân bằng 50%
Positive - 50% Negative.



Tiền xử lý

Từ ngữ được mã hóa thành số nguyên. Dùng `num_words=10000` để giữ 10.000 từ phổ biến nhất, loại bỏ từ hiếm.

Tải và Khám phá tập dữ liệu IMDb



Mỗi bài đánh giá đã được Keras mã hóa sẵn thành chuỗi số nguyên (integer index). Ta không phải xử lý văn bản thô (raw text) từ đầu.

```
from keras.datasets import imdb

# Tải 10000 từ phổ biến nhất
(train_data, train_labels), (test_data, test_labels) =
    imdb.load_data(num_words=10000)

# Kiểm tra dòng dữ liệu đầu tiên
print(train_data[0])
# [1, 14, 22, 16, 43, 530, 973, 1622, ... 178, 32]

print(train_labels[0])
# 1 (Tích cực)
```

Chuẩn bị Dữ liệu (Vector hóa)



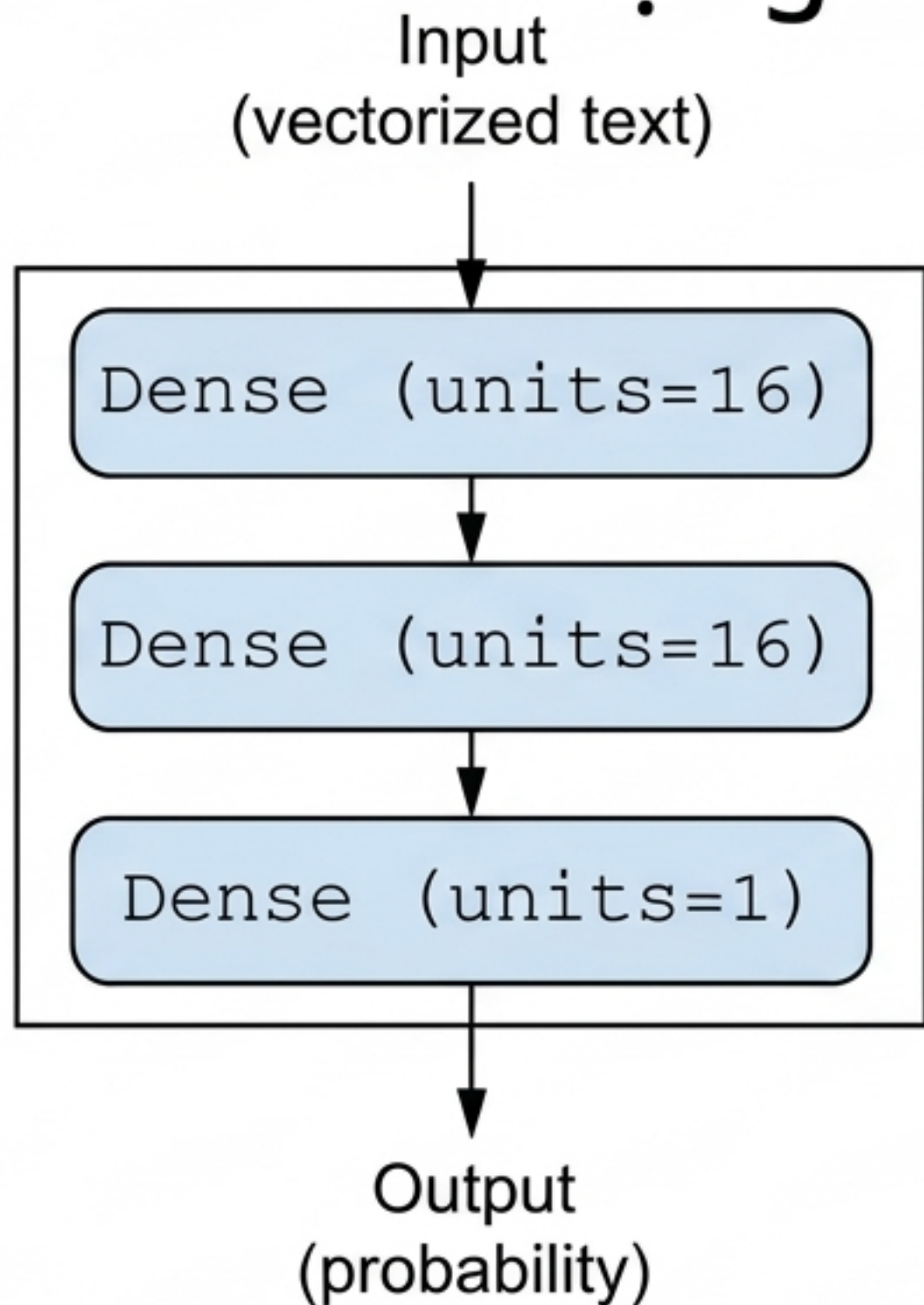
Sử dụng Multi-hot Encoding để biến chuỗi có độ dài thay đổi thành Vector 10000 chiều.

```
import numpy as np

# Vector hóa input (sử dụng hàm multi_hot_encode)
x_train = multi_hot_encode(train_data, num_classes=10000)
x_test = multi_hot_encode(test_data, num_classes=10000)

# Label đã là số nguyên 0 hoặc 1, ta chỉ cần chuyển sang float32
y_train = train_labels.astype("float32")
y_test = test_labels.astype("float32")
```

Kiến trúc Mạng Nơ-ron 3 Lớp



- **Cấu trúc:** Mạng có 2 lớp ẩn (Hidden Layers), mỗi lớp 16 nơ-ron (units). 1 lớp đầu ra (Output Layer) có 1 nơ-ron để dự đoán xác suất vô hướng.

- **Ý nghĩa:**

- Lớp 1: Trích xuất đặc trưng cơ bản.
- Lớp 2: Phân tích quan hệ phức tạp.
- Lớp cuối: Tổng hợp kết quả.

Hình 4.1: Cấu trúc mạng 3 lớp cho phân loại IMDb

Định nghĩa Mô hình trong Keras

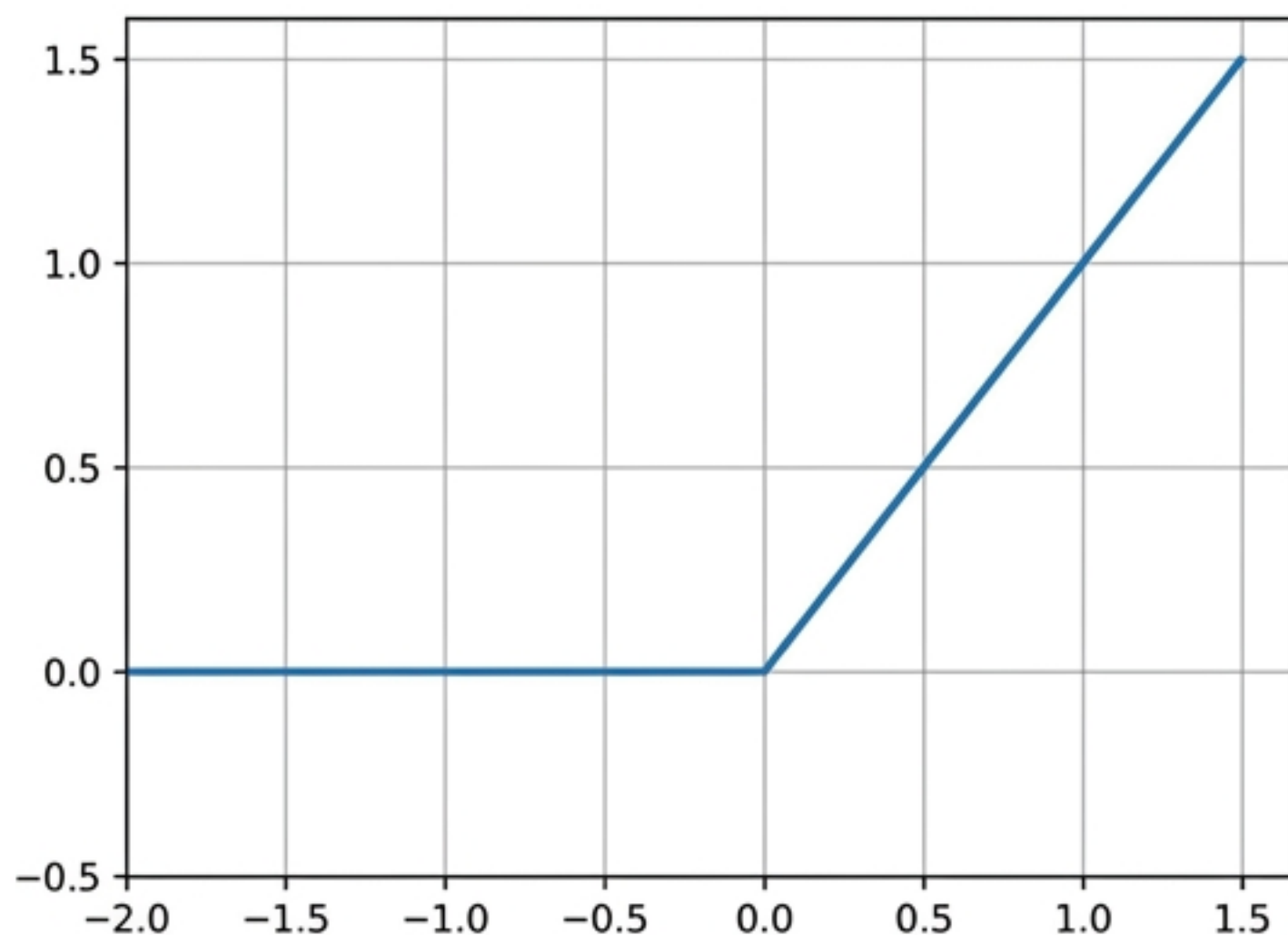


Ta sử dụng `keras.Sequential` để chồng các lớp liên tiếp nhau. Lớp `Dense` 16 units cần một hàm kích hoạt phi tuyến tính.

```
import keras
from keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```

Hàm kích hoạt (Activation): ReLU

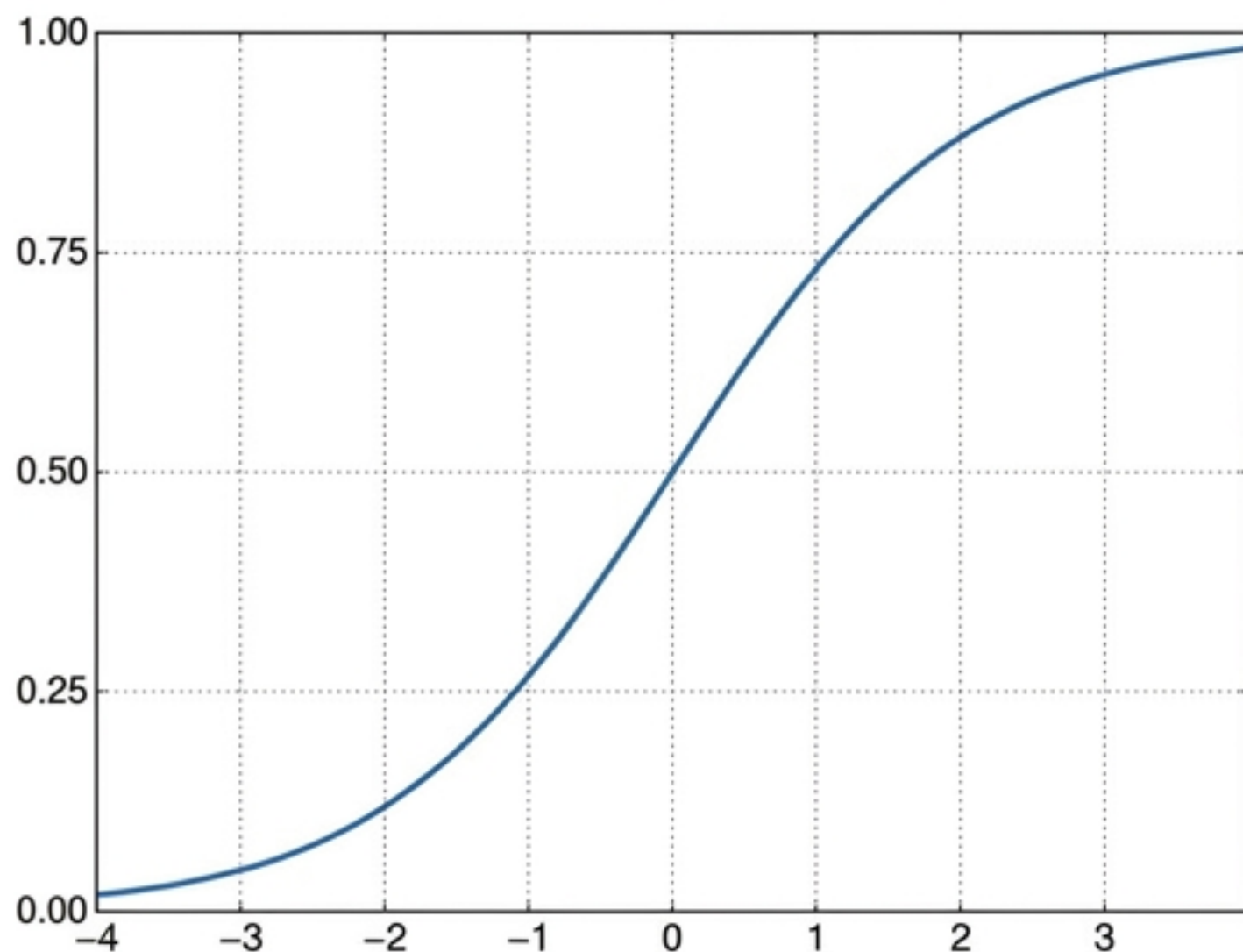


Hình 4.2: Đồ thị của hàm ReLU

$$f(x) = \max(0, x)$$

- **Cơ chế:** Loại bỏ giá trị âm (trả về 0), giữ nguyên giá trị dương.
- **Vai trò cốt lõi:** Nếu không có hàm kích hoạt phi tuyến, mạng nhiều lớp cũng chỉ tương đương 1 lớp tuyến tính, không thể giải quyết bài toán phức tạp.

Lớp Output: Hàm Sigmoid

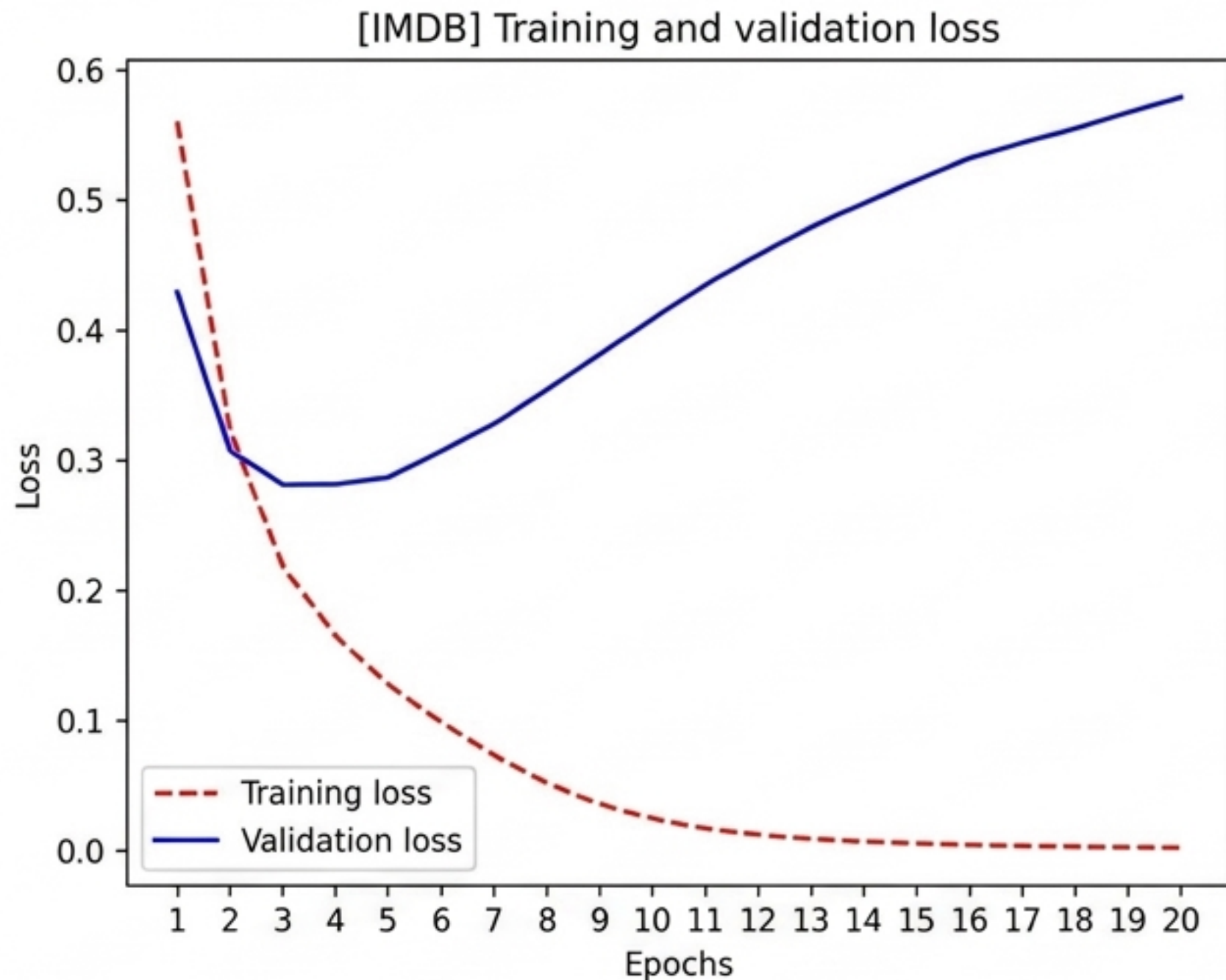


$$f(x) = \frac{1}{1 + e^{-x}}$$

- **Đầu ra** của mạng có thể là số bất kỳ $(-\infty, +\infty)$.
- **Sigmoid**: Ép bất kỳ giá trị nào về miền xác suất $[0, 1]$.
- **Ví dụ**: Kết quả 0.9 $\rightarrow$ Mô hình có 90% tự tin đây là bình luận Tích cực.

Biên dịch, Đánh giá và Hiện tượng Overfitting

```
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])  
history = model.fit(x_train, y_train, epochs=20, batch_size=512, validation_split=0.2)
```



Hình 4.4: Sự thay đổi Loss qua các Epochs



- **Training Loss (Đường chấm bi):** Liên tục giảm → Đang học rất tốt dữ liệu đào tạo.
- **Validation Loss (Đường liền):** Giảm đến epoch 4, sau đó đột ngột tăng lại.
- **Overfitting (Quá khớp):** Mô hình học thuộc lòng dữ liệu train nhưng không có khả năng tổng quát hóa dữ liệu chưa từng thấy.